Super Mario Bros.

eslem yildirim

April 2026

Gameplay Video: https://www.youtube.com/shorts/45tT$_y bjefUL$

1 Introduction

This report describes the implementation of a 2D platform game inspired by Super Mario Bros. The game includes core features such as movement physics, collision detection, level progression, and teleportation mechanics. It is developed in Java using the `StdDraw` library for graphics and user input.

2 Game Dynamics

The game operates on a continuous game loop managed by the `Game` class. This architecture divides responsibilities clearly among different classes, ensuring a smooth flow between player inputs, physics calculations, and screen rendering. Simple object-oriented structure used with the following main classes: Main, Game, Map, Mario, Enemy, and Level.

2.1 Core Mechanics

Collision Detection and Physics: The game uses the Axis-Aligned Bounding Box (AABB) method for collision detection, handled entirely by the `Map` class. The `checkCollision` method calculates if Mario's boundaries overlap with any obstacles, pipes, portals, keys, or locks. Gravity is applied to Mario constantly in the `Game` class. Before moving Mario downwards, the `isOnGround` method checks for floor collisions. If Mario hits a ceiling while jumping, `isOnCeiling` halts his upward speed. Wall collisions are prevented by the `dontTouchSideOfObstacleOrWorldFrame` method, which stops Mario from moving through solid blocks.

Level Mechanics and Progression: Levels are managed using the `Level` class, which acts as a data container holding specific enemies, coins, obstacles, keys, and locks. The `Game` class orchestrates the progression. When Mario reaches the exit pipe (`isAtExit`), the game checks if all required conditions (like collecting all coins) are met. Deep copying (`copy2DArray`) is used to load fresh copies of level elements. This ensures that if the player dies or restarts (pressing 'R'), the coins, keys, and destroyed locks return to their original states without memory reference issues.

Teleportation: Portals are connected in pairs (upper and lower). The `handleTeleport` method in the `Map` class manages this logic. Mario can enter the upper portal if he is

falling into it (downward speed -¿ jumping into). To enter the lower portal, the player must press the crouch key ('S'). Upon teleportation, Mario is instantly moved to the linked portal's coordinates. A boolean flag (`teleported`) is used to prevent infinite teleportation loops, allowing Mario to safely exit the portal zone.

Camera Mode: The camera system offers two visual experiences. By default, the camera shows the entire map (static mode). By pressing 'C', the dynamic camera mode is activated. In this mode, the `StdDraw.setXscale` and `setYscale` functions are updated dynamically based on Mario's coordinates. To prevent showing empty areas outside the map, the camera's center coordinates (`camX` and `camY`) are arranged to the boundaries of the 800×800 canvas.

2.2 Class Interactions and UML Diagrams

The system relies on strong interaction between the controller (`Game`), the environment (`Map` and `Level`), and the entities (`Mario` and `Enemy`). The `Game` class continuously calls `Mario.handleInput()` to get user actions, applies gravity, and then checks the environment using `Map` methods. Finally, it triggers `draw()` methods for all objects.

Table 1: UML Diagram of the Mario Class (Public vs. Private)

Public (+)	**Private (-)**
+ x: int	- spawnX: int = 60 *(final)*
+ y: int	- spawnY: int = 770 *(final)*
+ speedY: double	- size: int = 40 *(final)*
+ gravity: double = 2	- deathCount: int
	- isDead: boolean
	- faceRight: boolean
	- state: int
	- currentFrame: int
Methods (Metotlar)	
+ Mario(x: int, y: int)	
+ getSize(): int	
+ getDeathCount(): int	
+ getIsDead(): boolean	
+ handleInput(frame: int, onGround: boolean): int	
+ jump(): void	
+ checkEnemyCollision(enemy: Enemy): boolean	
+ die(): void	
+ respawn(): void	
+ draw(): void	

Table 2: UML Diagram of the Enemy Class (Public vs. Private)

Public (+)	**Private (-)**
Attributes	
	- x, y: double - size: int - x1, x2: double - speed: double = 2.0 - direction: int - type: String - bullets: ArrayList¡int[]¿ - shootPeriod: int = 0
Methods	
+ Enemy(x: double, y: double, size: int, direction: int, x1: double, x2: double, type: String) + Enemy(x: double, y: double, size: int, direction: int, x1: double, x2: double, type: String, bullets: ArrayList¡int[]¿) + getX(): double + getY(): double + getSize(): int + getType(): String + getBullets(): ArrayList¡int[]¿ + move(): void + draw(): void	

Table 3: UML Diagram of the Level Class (Public vs. Private)

Public (+)	**Private (-)**
Attributes	
	- id: int - enemies: ArrayList¡Enemy¿ - clue: String - coins: int[][] - obstacles: int[][] - keys: int[][] - locks: int[][]
Methods	
+ Level(id: int, enemies: ArrayList¡Enemy¿, clue: String, coins: int[][]) + Level(id: int, enemies: ArrayList¡Enemy¿, clue: String, coins: int[][], obstacles: int[][], keys: int[][], locks: int[][]) + getId(): int + getEnemies(): ArrayList¡Enemy¿ + getClue(): String + getCoins(): int[][] + getObstacles(): int[][] + getKeys(): int[][] + getLocks(): int[][] + updateCoins(keys: int[][]): void + updateKeys(keys: int[][]): void + updateLocks(locks: int[][]): void	

Table 4: UML Diagram of the Map Class (Public vs. Private)

Public (+)	**Private (-)**
Attributes	
+ correctedY: int + correctedX: int + hasKey: boolean = false	- obstacles: int[][] - pipes: int[][] - portals: int[][] - coins: int[][] - keys: int[][] - locks: int[][] - collisionIndex: int - teleported: boolean = false - marioRed: Color - marioGold: Color
Methods	
+ Map(obstacles: int[][], pipes: int[][], portals: int[][]) + updateObstacles(obstacles: int[][]): void + updateKeys(keys: int[][]): void + updateLocks(locks: int[][]): void + getObstacles(): int[][] + getPipes(): int[][] + getPortals(): int[][] + getCoins(): int[][] + checkCollision(nextX: double, nextY: double, entityHalfSize: double, elements: int[][]): boolean + isOnGround(x: int, y: int, halfSize: double, speedY: double, worldW: int, worldH: int): boolean + isOnCeiling(x: int, y: int, halfSize: double, speedY: double, worldW: int, worldH: int): boolean + dontTouchSideOfObstacleOrWorldFrame(nextX: int, y: int, halfSize: double, worldW: int): int + setCoins(coins: int[][]): void + collectCoin(mario: Mario): int + allCoinsCollected(): boolean + handleTeleport(mario: Mario, sPressed: boolean, speedY: double): void + isAtExit(mario: Mario): boolean + drawObstacles(): void + drawPipes(): void + drawPortals(): void + drawCoins(): void + drawLocks(): void + drawKeys(): void	

Table 5: UML Diagram of the Game Class (Public vs. Private)

Public (+)	**Private (-)**
Attributes	
	- levels: ArrayList¡Level¿ - map: Map - width: int - height: int - fps: int = 16 - pipes, obstacles0, keys0, locks0, coins0: int[][] - skyColor, mainMenuColor0, mainMenuColor1, mainMenuColor2, marioDarkBlue, marioRed, marioGold, marioLightBlue: Color
Methods	
+ Game(levels: ArrayList¡Level¿, map: Map) + run(): void	- applyGravity(mario: Mario): void - startScreen(mario: Mario): void - endScreen(elapsedTime: String, deathCount: int): int - getTime(startingTime: long): String - drawHUD(currentLevel: Level, mario: Mario, gameStartTime: long, isCameraFollowing: boolean, camX: double, camY: double): void - levelBanner(id: int, currentLevel: Level, currentEnemies: ArrayList¡Enemy¿, mario: Mario, elapsedTime: String, isCameraFollowing: boolean, camX: double, camY: double): long - copy2DArray(original: int[][]): int[][]

2.3 Gameplay Screenshots

Below are screenshots demonstrating the level progression and the camera system in action.

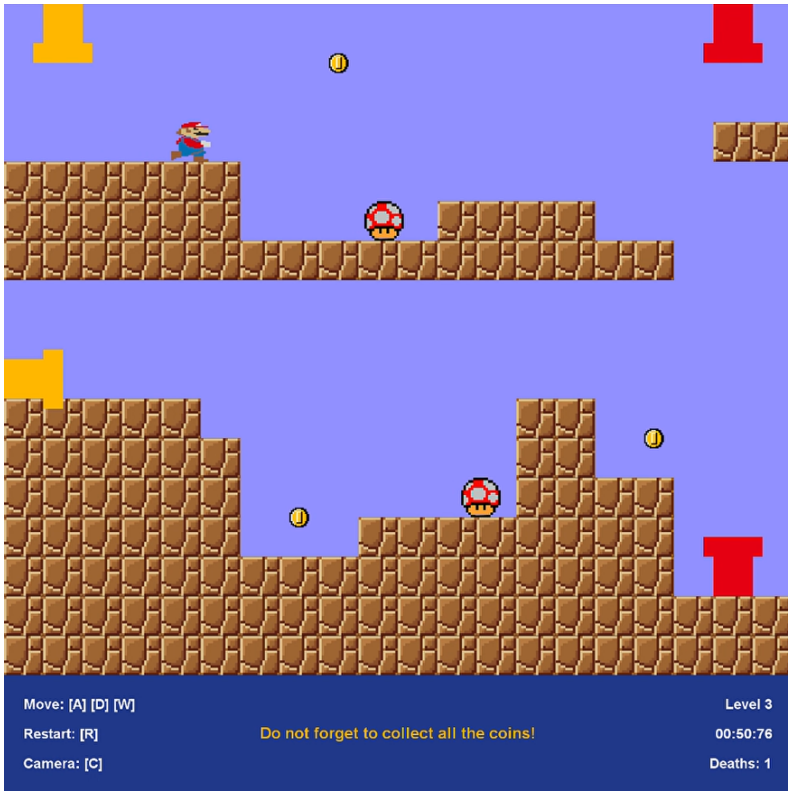


Figure 1: Gameplay overview from Level 3 showing enemies and coins with standard camera view.

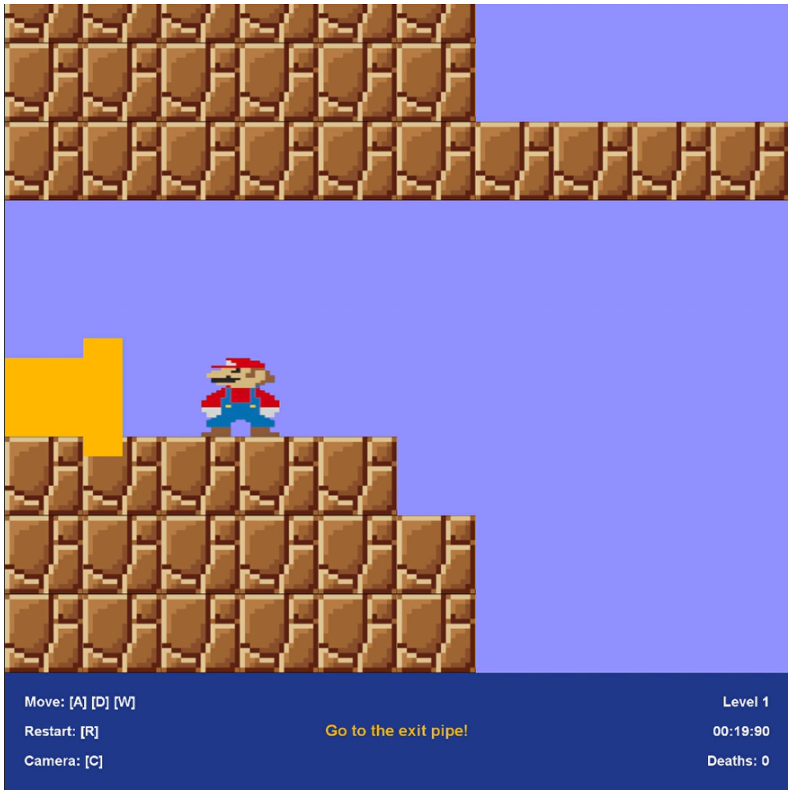


Figure 2: Dynamic camera mode focusing on Mario, keeping the action centered.

3 Advanced Mechanics and Puzzle Design

The final level (level 4) introduces new puzzle mechanics and hazard types to increase the game's complexity.

Cannons and Bullets: Unlike standard enemies, cannons are stationary hazards that generate moving bullet objects. These projectiles travel linearly and require precise timing to avoid.

Key and Lock Mechanics: A new progression blocker is introduced. The player must locate and collect a Key object, which updates the HUD. Holding the key allows the player to interact with and permanently remove Lock blocks from the map, opening new pathways to the exit pipe.

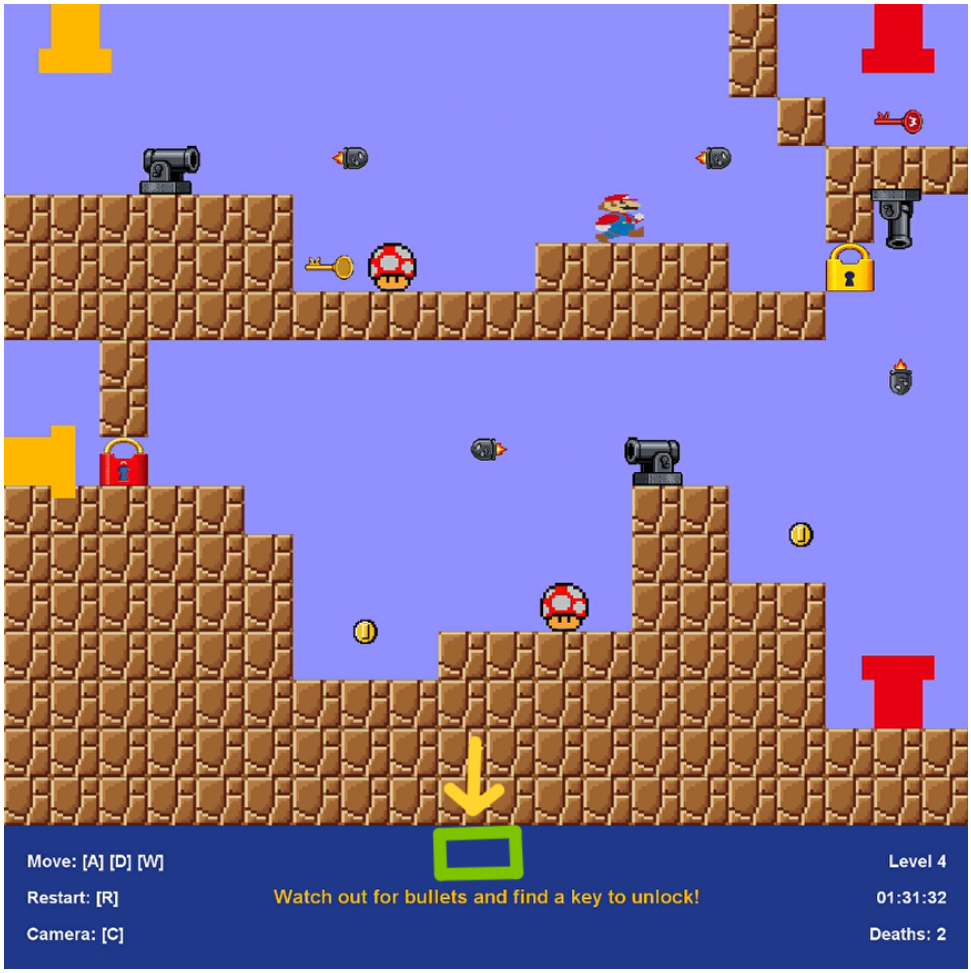


Figure 3: Final Level showcasing cannons, bullets, and the key-lock system.



Figure 4: The HUD displays a key icon (highlighted) when the player collects a key.

Table 6: Final Level New Mechanics Summary

Element	**Description**
Cannon	Stationary block that fires continuous projectiles.
Bullet	Linear projectile that causes player death upon collision.
Key	Collectible item required to open specific barriers.
Lock	Destructible barrier that disappears when touched by a player holding a key.